

Question 1

How does Shor's algorithm threaten the security of RSA and Elliptic Curve Cryptography (ECC), and what are the potential consequences for current digital infrastructure?

Ans:

Shor's algorithm, developed by mathematician Peter Shor in 1994, is a quantum algorithm capable of solving the integer factorization problem and discrete logarithm problem in polynomial time.

RSA security is based on the computational difficulty of factoring large integers, and ECC relies on the hardness of the elliptic curve discrete logarithm problem. On

classical computers, these problems require exponential time for sufficiently large keys, making them secure in practice.

However, Shor's algorithm can solve both problems exponentially faster on a quantum computer, reducing the complexity from sub-exponential or exponential to polynomial time. This means that once sufficiently powerful quantum computers are available, they can efficiently break RSA and ECC encryption, rendering these systems obsolete.

Potential consequences include:

1. Data Breaches: Encrypted sensitive information such as banking data, health records, and classified government documents could be

decrypted.

2. Digital Signature Forgery:

Attackers could forge digital certificates and signatures, compromising authentication mechanisms.

3. Retrospective Decryption: Encrypted communications intercepted and stored today could be decrypted in the future ("harvest now, decrypt later" attack).

4. Infrastructure Risk: Critical systems such as financial transactions, secure communications, and identity verification systems would require urgent migration to post-quantum cryptography.

Question 2

Discuss the role of quantum key distribution (QKD) in future cryptographic systems. How does it differ from classical public-key encryption?

Quantum Key Distribution (QKD)

enables two parties to generate and share a secret key securely, with security guaranteed by quantum mechanics rather than computational hardness. The most well-known protocol, BB84, uses the principle that any attempt to measure quantum states disturbs them, alerting parties to the presence of an eavesdropper.

Differences from classical public-key encryption:

1. Security Basis: QKD relies on physical laws (quantum mechanics)

while classical PKI is based on mathematical problems like factoring or discrete logs.

2. Eavesdropping Detection: In QKD, interception alters the quantum state, revealing eavesdropping attempts.

3. Quantum Resistance: QKD is inherently secure against quantum computer attacks, unlike RSA/ECC.

4. Implementation Challenges: QKD requires specialized hardware like single-photon emitters and detectors, making it harder to deploy than software-based classical methods.

QKD is expected to play a key role in securing high-value communications in the quantum era, particularly in sectors like government, military, and financial services.

Question 3

What are the main differences between lattice-based cryptography and traditional number-theoretic approaches like RSA, particularly in the context of quantum resistance? Lattice-based cryptography builds its security on the hardness of lattice problems such as the Shortest Vector Problem (SVP) and Learning With Errors (LWE). These problems are believed to be resistant to both

classical and quantum attacks.

Differences:

1. Security Foundation: Lattice problems are NP-hard, while RSA relies on factoring and ECC on discrete logs, both solvable by Shor's algorithm.
2. Quantum Resistance: Lattice-based schemes are post-quantum secure, making them candidates for future cryptographic standards.
3. Advanced Capabilities: Lattice cryptography supports fully homomorphic encryption, identity-based encryption, and other advanced functionalities not easily achievable with RSA/ECC.
4. Performance and Key Sizes: Lattice schemes can be faster in

encryption/decryption, but often require larger key sizes than ECC (though smaller than equivalent RSA keys for high security levels).

In the quantum era, lattice-based systems are a promising replacement for RSA/ECC.

Question 4

Develop a Python-based PRNG that uses the current system time and a custom seed value. Write complete program and corresponding output.

Example Python Code:

```
python
```



```
import time
import random

# Create a custom seed using
system time and an arbitrary
constant
custom_seed = int(time.time())

987654321

random.seed(custom_seed)

# Generate random numbers
numbers = [random.randint(0, 100) for
_ in range(5)]

print("Seed used:", custom_seed)

print("Random numbers:", numbers)
```


Explanation:

- The `'time.time()'` function returns the current time in seconds since the epoch.
- XOR with a constant adds unpredictability.
- `'random.seed()'` initializes the random number generator.
- We generate a list of 5 random integers between 0 and 100.

Example Output:

Seed used: 1723719922

Random numbers: [23, 87, 5, 44, 91]

Question 5

Explain the Sieve of Eratosthenes algorithm and use it to find all prime numbers less than 50. How does its time complexity compare to trial division?

Ans:

Sieve of Eratosthenes Algorithm:
The Sieve of Eratosthenes is an efficient method to find all prime numbers up to a given limit.

Steps:

1. Write down all numbers from 2 to n .
2. Start with $p = 2$. Mark all multiples of p (except p) as composite.

3. Find the next unmarked number, set it as p , and mark all its multiples.

4. Repeat until $p > \sqrt{n}$.

Example for $n = 50$:

Initial list: 2, 3, 4, 5, ..., 50

After removing multiples of 2: 2, 3, 5, 7, 9, ...

Continue for 3, 5, 7.

Final primes < 50 : 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47.

Time Complexity:

- Sieve: $O(n \log \log n)$
- Trial Division: $O(n\sqrt{n})$

Thus, the Sieve is much faster for large n .

Question 6

State and explain the necessary and sufficient conditions for a composite number to be a Carmichael number. Then verify whether the number $n = 561$ and $n = 1105$ and $n = 1729$ are Carmichael numbers.

Ans:

Definition: A Carmichael number is a composite integer n that satisfies the congruence $b^{n-1} \equiv 1 \pmod{n}$ for all integers b that are coprime to n .

These are 'Fermat pseudoprimes' to all bases coprime to them.

Necessary and Sufficient Conditions (Korselt's Criterion):

A composite number n is a Carmichael number if and only if:

1. n is square-free (no repeated prime factors).

2. For every prime divisor p of n , $p-1$ divides $n-1$.

Verification:

- $n = 561$: Factors into $3 \times 11 \times 17$. It is square-free.

Check divisibility:

$p=3 \rightarrow p-1=2$ divides $560 \rightarrow \text{Yes}$.

$p=11 \rightarrow p-1=10$ divides $560 \rightarrow \text{Yes}$.

$p=17 \rightarrow p-1=16$ divides 560 $\rightarrow$ Yes.

561 is Carmichael.

- $n = 1105$: Factors into $5 \times 13 \times 17$.

Square-free.

$p=5 \rightarrow p-1=4$ divides 1104 $\rightarrow$ Yes.

$p=13 \rightarrow p-1=12$ divides 1104 $\rightarrow$ Yes.

$p=17 \rightarrow p-1=16$ divides 1104 $\rightarrow$ Yes.

1105 is Carmichael.

- $n = 1729$: Factors into $7 \times$

13×19 . Square-free.

$p=7 \rightarrow p-1=6$ divides 1728 $\rightarrow$ Yes.

$p=13 \rightarrow p-1=12$ divides 1728 $\rightarrow$

Yes.

$p=19 \rightarrow p-1=18$ divides $1728 \rightarrow$ Yes.

1729 is Carmichael.

Question 7

Determine whether the following are valid algebraic structures and

justify your answer: (a) Is the set

$\mathbb{Z}_{11}$ with operations $(+, \cdot)$ a ring?

(b) Are the sets $(\mathbb{Z}_{37}, +)$ and

$(\mathbb{Z}_{35}, \times)$ Abelian groups?

Ans:

Part (a): $\mathbb{Z}_{11}$ denotes integers modulo 11. Under addition and multiplication modulo 11:

- Closure: Addition and multiplication of two elements mod 11 stays in Z_{11} .
- Associativity: Both operations are associative.
- Additive identity: 0 exists.
- Multiplicative identity: 1 exists.
- Additive inverses exist: For every a , there is a b such that $a+b \equiv 0 \pmod{11}$.
- Distributive laws hold.

Z_{11} with $(+, \cdot)$ forms a commutative ring and, since 11 is prime, even a field.

Part (b):

- $(Z_{37}, +)$: Integers modulo 37 under addition form an Abelian

group because addition mod n is associative, commutative, identity exists (0), inverses exist.

Abelian group.

- $(\mathbb{Z}_{35}, \times)$: Multiplication modulo 35 does not form a group for all elements since not all elements have inverses (e.g., 5 has no inverse mod 35).

Not an Abelian group under multiplication for the entire set.

Question 8

What is the remainder when -52 is reduced modulo 31?

We compute $(-52) \bmod 31$.

First, find how many times 31 fits

into -52: $-52 \div 31 \approx -1$ remainder
-21.

Since remainder must be non-negative, add 31: $-21 + 31 = 10$.

Remainder = 10.

Question 9

Determine the multiplicative inverse of $7 \pmod{26}$, if it exists. (Use extended Euclidean algorithm).

Ans:

We solve $7x \equiv 1 \pmod{26}$.

Using Extended Euclidean Algorithm:

$$26 = 3 \times 7 + 5$$

$$7 = 1 \times 5 + 2$$

$$5 = 2 \times 2 + 1$$

Back substitute:

$$1 = 5 - 2 \times 2$$

$$= 5 - 2 \times (7 - 1 \times 5) = 3 \times 5 - 2 \times 7$$

$$= 3 \times (26 - 3 \times 7) - 2 \times 7$$

$$= 3 \times 26 - 11 \times 7$$

$$\text{Thus, } -11 \times 7 \equiv 1 \pmod{26} \rightarrow x$$

$$\equiv -11 \equiv 15 \pmod{26}.$$

Multiplicative inverse = 15.

Question 10

Evaluate $(-8 \times 5) \bmod 17$, and explain how to simplify negative modular multiplication.

First, compute the product: $(-8 \times 5) = -40$.

Now reduce modulo 17:

$-40 \div 17 = -3$ remainder 11 (since $-40 + 51 = 11$).

Result = 11.

Simplification for negative modular multiplication:

You can first reduce negative numbers modulo n to get a positive equivalent, then multiply.

Example: $-8 \bmod 17 = 9$. Then $(9 \times 5) \bmod 17 = 45 \bmod 17 = 11$.

Question 11

State and prove Bezout's Theorem.
Use it to find the multiplicative inverse of 97 modulo 385.

Bezout's Theorem: For any integers a and b , there exist integers x and y such that $ax + by = \gcd(a, b)$.

Ans:

Proof Idea: The gcd can be expressed as a linear combination of a and b by the Extended Euclidean Algorithm.

To find inverse of 97 mod 385:

$\gcd(97, 385) = 1$, so inverse exists.

Using Extended Euclidean Algorithm:

$$385 = 3 \times 97 + 94$$

$$97 = 1 \times 94 + 3$$

$$94 = 31 \times 3 + 1$$

Back substitute:

$$1 = 94 - 31 \times 3$$

$$= 94 - 31 \times (97 - 1 \times 94) = 32 \times 94 - 31 \times 97$$

$$= 32 \times (385 - 3 \times 97) - 31 \times 97$$

$$= 32 \times 385 - 127 \times 97$$

Thus, $-127 \times 97 \equiv 1 \pmod{385} \rightarrow$

$$\text{inverse} \equiv 258 \pmod{385}.$$

$$\text{Multiplicative inverse} = 258.$$

Question 12

Using Bezout's identity, prove the equation $ax + by = \gcd(a, b)$ has integer solutions. Find x such that $43x \equiv 1 \pmod{240}$.

Ans:

Bezout's identity states there are integers x, y such that

$$ax + by = \gcd(a, b).$$

Proof uses Extended Euclidean Algorithm.

$$\text{For } 43x \equiv 1 \pmod{240}:$$

$$\gcd(43, 240) = 1 \rightarrow \text{inverse exists.}$$

$$240 = 5 \times 43 + 25$$

$$43 = 1 \times 25 + 18$$

$$25 = 1 \times 18 + 7$$

$$18 = 2 \times 7 + 4$$

$$7 = 1 \times 4 + 3$$

$$4 = 1 \times 3 + 1$$

Back substitute:

$$1 = 4 - 1 \times 3$$

$$= 4 - 1 \times (7 - 1 \times 4) = 2 \times 4 - 1 \times 7$$

$$= 2 \times (18 - 2 \times 7) - 1 \times 7$$

$$= 2 \times 18 - 5 \times 7$$

$$= 2 \times (43 - 1 \times 25) - 5 \times (25)$$

$$= 2 \times 43 - 7 \times 25$$

$$= 2 \times 43 - 7 \times (240 - 5 \times 43)$$

$$= 37 \times 43 - 7 \times 240$$

$$\text{Thus, } 37 \times 43 \equiv 1 \pmod{240}.$$

$$x = 37.$$

Question 13

State Fermat's Little Theorem and explain how it is used to test for primality. Is 561 a prime number based on this test? Evaluate $5^{123} \pmod{175}$ using Fermat's Little Theorem.

Ans:

Fermat's Little Theorem: If p is prime and $\gcd(a, p) = 1$, then $a^{(p-1)} \equiv 1 \pmod{p}$.

For primality test: Choose random a
 $< n$, check if $a^{(n-1)} \equiv 1 \pmod{n}$.
 If not, n is composite.

Test for $n = 561$ with $a = 2$:

$$2^{560} \bmod 561 = 1 \text{ (passes for some}$$

a) but 561 is known composite

(Carmichael). So FLT can give false
 positives.

Evaluate $5^{123} \bmod 175$:

Since $175 = 25 \times 7$, use Chinese

Remainder Theorem.

$$\text{Mod } 25: 5^{123} \equiv 0.$$

$$\text{Mod } 7: 5^6 \equiv 1, \text{ so } 5^{123} \equiv 5$$

$$(6 \times 20 + 3) \equiv (120 + 3) \equiv 125 \equiv$$

$$6 \pmod{7}.$$

$$\text{CRT: } x \equiv 0 \pmod{25}, x \equiv 6 \pmod{7}$$

$$7 \rightarrow x = 125.$$

$$\text{Answer} = 125.$$

Question 14

State and prove the Chinese Remainder Theorem. Then solve $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$, $x \equiv 2 \pmod{7}$.

Ans:

Chinese Remainder Theorem (CRT):

If $n_1, n_2, \dots, n_k$ are pairwise coprime, then the system of congruences has a unique solution

modulo $N = n_1 n_2 \dots n_k$.

Proof idea: Construct solution by successive substitutions or by explicit formula using modular inverses.

Solve:

$$N = 3 \times 5 \times 7 = 105.$$

$$N_1 = 35, \quad N_2 = 21, \quad N_3 = 15.$$

Inverses:

$$35\gamma \equiv 1 \pmod{3} \rightarrow \gamma = 2.$$

$$21\gamma \equiv 1 \pmod{5} \rightarrow \gamma = 1.$$

$$15\gamma \equiv 1 \pmod{7} \rightarrow \gamma = 1.$$

Solution:

$$x = 2 \times 35 \times 2 + 3 \times 21 \times 1 + 2 \times 15 \times 1 = 140 + 63$$

$$+ 30 = 233.$$

$$233 \bmod 105 = 23.$$

$$x \equiv 23 \bmod 105.$$

Question 15

Briefly explain the CIA triad in information security. How does each component contribute to building a secure system?

Ans:

CIA Triad: Confidentiality,
Integrity, Availability.

- Confidentiality: Ensuring information is accessible only to

authorized parties. Achieved via encryption, access controls.

- Integrity: Ensuring data is accurate and unaltered. Achieved with hashing, digital signatures.
- Availability: Ensuring systems /data are accessible when needed.

Achieved via redundancy, backups, DoS protection.

All three must be balanced for a secure system.

Question 16

How does steganography differ from cryptography in the context of information security, and what are

common techniques used for hiding data in digital media?

Ans:

Cryptography: Protects data by transforming it into unreadable form without a key.

Steganography: Hides the existence of the message by embedding it in other data (e.g., images, audio).

Common techniques:

- LSB (Least Significant Bit)

insertion in images.

- Audio echo hiding.

- Video frame manipulation.

Steganography can be combined with cryptography for added security.

Question 17

What are the key differences between phishing, malware, and denial-of-service (DoS) attacks in terms of their method and impact on system security?

Ans:

- Phishing: Social engineering attack to trick users into revealing credentials. Delivered via email, fake sites.
- Malware: Malicious software that disrupts, damages, or gains unauthorized access (e.g., viruses, ransomware).
- DoS: Overwhelms a service with traffic, rendering it unavailable.

Methods differ (human deception vs code execution vs resource flooding) but all compromise security.

Question 18

Explain how legal frameworks such as the General Data Protection Regulation (GDPR) help mitigate cyber attacks and protect user privacy.

Ans:

GDPR enforces strict rules on data collection, processing, and storage:

- Requires consent for data collection.

- Right to access, connect, delete personal data.
- Mandatory breach notification within 72 hours.
- Heavy penalties for non-compliance.

It indirectly mitigates cyber attacks by enforcing strong data protection and accountability measures.

Question 19

Explain the basic working of the DES algorithm using a simple 64-bit plaintext block and a 56-bit key.

Ans:

DES (Data Encryption Standard) is a symmetric block cipher:

1. Initial Permutation (IP) of plaintext.
2. Split into L_0 and R_0 (32 bits each).
3. 16 rounds: $R_i = L_{i-1} \text{ XOR } f(R_{i-1}, K_i)$, $L_i = R_{i-1}$.
4. f -function: Expansion (32 $\rightarrow$ 48 bits), XOR with K_i , S-box substitution (48 $\rightarrow$ 32), permutation.
5. Final swap and inverse IP.

Key schedule generates 16 subkeys from the 56-bit key.

Question 20

In the DES algorithm, given
 $L_0 = 0xF0F0F0F0$, $R_0 = 0x0F0F0F0F$, round
 key $K_1 = 0x0F0F0F0F$, compute first
 round output.

Ans:

$$f(R_0, K_1) = R_0 \text{ XOR } K_1 = 0x0F0F0F0F \\
\text{XOR } 0x0F0F0F0F = 0x00000000.$$

$$\text{Then } L_1 = R_0 = 0x0F0F0F0F, R_1 = L_0 \\
\text{XOR } f(R_0, K_1) = 0xF0F0F0F0 \text{ XOR } \\
0x00000000 = 0xF0F0F0F0.$$

Question 21

Given the input word $[0x23, 0xA7, 0x4C, 0x19]$. Use the following partial AES S-box to perform the SubBytes transformation. Provide the resulting output word.

Row ↓ \ Col →	3	4	7	9	A	C
1	6D	.	.	C6	.	.
2	D4	.	.	.	.	.
4	.	A1	.	.	.	2E
A	.	.	63	.	D2	.

Ans:

Apply the AES S-box to each byte independently ($0xXY$ indexes row X , column Y in hex). Using the standard AES S-box:

$$S(0x23) = 0x26$$

$$S(0xA7) = 0x5C$$

$$S(0x4C) = 0x29$$

$$S(0x19) = 0xD4$$

Output word after SubBytes: $[0x26, 0x5C, 0x29, 0xD4]$.

Question 22

In AES encryption, apply the AddRoundKey step only. Given: Input word: $[0x1A, 0x2B, 0x3C, 0x4D]$; Round key word: $[0x55, 0x66, 0x77, 0x88]$.

Perform XOR between the input word and the round key, and write the resulting output word.

Ans:

AddRoundKey is a byte-wise XOR:

$$0x1A \oplus 0x55 = 0x4F$$

$$0x2B \oplus 0x66 = 0x4D$$

$$0x3C \oplus 0x77 = 0x4B$$

$$0x4D \oplus 0x88 = 0xC5$$

Output: $[0x4F, 0x4D, 0x4B, 0xC5]$.

Question 23

Show how AES MixColumns uses the fixed matrix over $GF(2)$:

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

to transform an input column. Use an example column with values $[0x01, 0x02, 0x03, 0x04]$.

Ans:

In AES, MixColumns multiplies each state column by the fixed matrix over $GF(2^8)$ (with the Rijndael polynomial). For input column $a = [a_0, a_1, a_2, a_3] = [01, 02, 03, 04]$, the output r is:

$$r_0 = (02 \cdot a_0) \oplus (03 \cdot a_1) \oplus (01 \cdot a_2)$$

$$\oplus (01 \cdot a_3)$$

$$r_1 = (01 \cdot a_0) \oplus (02 \cdot a_1) \oplus (03 \cdot a_2)$$

$$\oplus (01 \cdot a_3)$$

$$r_2 = (01 \cdot a_0) \oplus (01 \cdot a_1) \oplus (02 \cdot a_2)$$

$$\oplus (03 \cdot a_3)$$

$$r_3 = (03 \cdot a_0) \oplus (01 \cdot a_1) \oplus (01 \cdot a_2)$$

$$\oplus (02 \cdot a_3)$$

Computing in $GF(2^8)$ gives:

$$r_0 = 0x03, \quad r_1 = 0x04, \quad r_2 = 0x09, \quad r_3 = 0x0A.$$

$$\text{So } \text{MixColumns}([01, 02, 03, 04]) = [0x03, 0x04, 0x09, 0x0A].$$

Question 24

Describe how AES-OFB mode works. How does it ensure synchronization between encryption and decryption streams?

Ans:

OFB (Output Feedback) mode turns a block cipher into a synchronous stream cipher. It repeatedly encrypts a shift register (starting from the IV) to generate a keystream, and XORs that keystream with plaintext to produce ciphertext:

$$S_0 = IV$$

$$Z_1 = E_K(S_0); \quad C_1 = P_1 \oplus Z_1$$

$$Z_2 = E_K(Z_1); \quad C_2 = P_2 \oplus Z_2$$

... and so on.

Decryption uses the same keystream:

$$P_1 = C_1 \oplus Z_1, \quad P_2 = C_2 \oplus Z_2, \quad \dots$$

Synchronization: Both sides must share the same key and IV. Because the keystream is independent of the plaintext/ciphertext (it depends only on the previous keystream block),

errors do not propagate beyond the corrupted bits, and resynchronization is possible if IVs and block counters are aligned.

Question 25

Which AES modes cause error propagation during decryption, and how does this affect the integrity

of the decrypted message?

Illustrate with CBC and CFB modes.

Ans:

- CBC (Cipher Block Chaining): A 1-bit error in ciphertext block C_j corrupts the entire plaintext block P_j during decryption, and flips the corresponding bit in the next block P_{j+1} due to XOR with C_j . Thus, errors propagate to two blocks.
- CFB (Cipher Feedback): A bit flip in C_j causes a bit flip in P_j only at the same position; subsequent blocks are unaffected because the keystream is generated by encrypting prior ciphertext, not the plaintext.
- CTR and OFB: Behave like

stream ciphers; a bit error affects only the corresponding bit of plaintext, no further propagation.

Integrity implication: None of these modes by themselves provide integrity; they only provide confidentiality. Use an authenticated mode (e.g., GCM) or add a MAC to detect tampering. CBC reveals tampering via widespread gibberish, but does not detect modifications reliably without a MAC.

Question 26

Which AES mode would you recommend for encrypting large files with parallel processing, and why?

Justify your choice between ECB, CBC, and CTR.

Recommendation: CTR (Counter) mode.

Ans:

- Parallelism: CTR encrypts independent counters (nonce counter) that can be processed in parallel for both encryption and decryption, giving excellent throughput on multicore CPUs and GPUs.
- Random access: Supports seeking/decrypting any block independently, ideal for large files.
- Security: Avoids ECB's pattern leakage. Unlike CBC, which chains blocks (encryption not parallelizable),

CTR's blocks are independent.

Why not ECB? Identical plaintext blocks yield identical ciphertext blocks (leaks patterns).

Why not CBC? Encryption is inherently serial (though decryption can be parallelized), making it slower for very large data sets. CTR must be used with unique nonces/counters to remain secure.

Question 27

Given a message "A" represented as

$M = 1$, encrypt it using public key

$e = 5$, $n = 14$. What is the

ciphertext? Then decrypt using
private key $d = 11$.

Ans:

RSA encryption: $C = M^e \bmod n = 15$
 $\bmod 14 = 1$.

Decryption: $M = C^d \bmod n = 1^{11} \bmod$
 $14 = 1$.

Ciphertext = 1. Recovered plaintext = 1

Question 28

Given: Message hash $H(M) = 5$, RSA

Private Key: $d = 3$, $n = 33$. Generate
the digital signature.

Ans:

RSA signature $s = H(M)^d \bmod n = 5$

$$3 \bmod 33 = 125 \bmod 33.$$

$$33 \times 3 = 99; 125 - 99 = 26.$$

Signature = 26. (Verification uses s

$e \equiv H(M) \pmod{n}$ with the public exponent e .)

Question 29'

Aleya and Badol are using the Diffie-Hellman key exchange protocol. They agree on the following public values: Prime $p = 23$; Base

(generator) $g=3$. Aleya chooses private

key $a=4$, Badol chooses private key $b=5$. Compute the public keys and the shared secret key.

Ans:

Public keys:

$$\text{Aleya: } A = g^a \text{ mod } p = 3^4 = 81 \equiv 12 \pmod{23}.$$

$$\text{Badol: } B = g^b \text{ mod } p = 3^5 = 243 \equiv 13 \pmod{23}.$$

Shared secret:

$$\begin{aligned} K &= B^a \text{ mod } p = 13^4 \text{ mod } 23. \quad 13^2 \\ &\equiv 169 \equiv 8 \pmod{23}; \quad 13^4 \equiv 8^2 = \\ 64 &\equiv 18 \pmod{23}. \end{aligned}$$

(Equivalent via $A b \bmod p = 125 \equiv 18.$)

So $A = 12$, $B = 13$, shared key $K = 18.$

Question 30

A simple (non-cryptographic) hash function $H(x)$ is defined as the sum of ASCII values of the characters in x , modulo 100: $H(x) = (\sum \text{ASCII}(x_i)) \bmod 100$. Compute $H("AB")$ and $H("BA")$. Do the two messages produce the same hash?

What does this imply about collision resistance?

Ans:

ASCII: 'A' = 65, 'B' = 66.

$$H("AB") = (65 + 66) \bmod 100 = 131 \bmod$$

$$100 = 31 .$$

$$H("BA") = (66 + 65) \bmod 100 = 131 \bmod$$

$$100 = 31 .$$

Yes, both produce the same hash.

This shows the function is not collision resistant, as distinct messages can easily collide (order-insensitive sum modulo 100).

Question 31

In a secure message system, a simple Message Authentication Code (MAC) is computed as $MAC = (Message + SecretKey) \bmod 17$. Given: Message = 15, SecretKey = 7. Compute the MAC.

Suppose an attacker changes the message to 10 but doesn't know the key. Can they forge the correct MAC easily? Explain briefly.

Ans:

$$MAC = (15 + 7) \bmod 17 = 22 \bmod 17 = 5$$

If the attacker observes $(M, MAC) = (15, 5)$, they can compute the key:

$\text{SecretKey} \equiv \text{MAC} - M \equiv 5 - 15 \equiv -10$
 $\equiv 7 \pmod{17}$. Then they can forge
 MAC' for $M' = 10$ as $(10 + 7) \pmod{17}$
 $= 0$.

Therefore, this MAC is insecure
 because it is linear and trivially
 reveals the key from one sample.

Real MACs (e.g., HMAC) avoid this
 weakness.

Question 32

Explain the steps involved in the
 TLS handshake process. How are
 symmetric keys established securely

using asymmetric cryptography during the handshake?

Ans:

Typical TLS 1.2 (ECDHE) handshake

flow:

1) ClientHello: Client sends supported cipher suites, TLS version, random nonce (ClientRandom), and extensions (e.g., SNI).

2) ServerHello: Server chooses parameters, returns its random nonce (ServerRandom), certificate chain, and ECDHE parameters (curve + ephemeral public key). Optionally requests client cert.

3) Key Exchange: Client verifies server certificate, then computes the ECDHE shared secret using server's ephemeral public key and its own ephemeral private key; sends its ephemeral public key.

4) Key Derivation: Both sides derive the premaster secret from ECDHE, then use a KDF (PRF) over ClientRandom and ServerRandom to produce symmetric keys and MAC keys.

5) Finished messages: Each side sends a Finished message (MAC over transcript) to authenticate the handshake and confirm keys.

Security comes from authenticated ECDHE (server identity via certificate) providing forward secrecy and resistance to passive eavesdropping. Asymmetric crypto authenticates the server; symmetric keys are derived from ECDHE shared secret.

Question 33

Explain the layered architecture (protocol stack) of SSH. Briefly describe the roles of each layer.

Ans:

SSH has three logical layers:

- 1) Transport Layer (ssh-transport):

Negotiates algorithms, performs key exchange (e.g., ECDH), provides server authentication, and sets up encryption/MAC for confidentiality and integrity.

2) User Authentication Layer (ssh-userauth): Authenticates the client user via passwords, public keys, or keyboard-interactive methods.

3) Connection Layer (ssh-connection): Multiplexes the encrypted tunnel into multiple logical channels (shell sessions, exec commands, port forwarding, SFTP), manages flow control and channel requests.

This layered design separates concerns and allows different authentication methods over a single secure transport.

Question 34

Explain the TLS handshake again, focusing on message flow and security properties (freshness, server authentication, and forward secrecy).

Ans:

Message flow (ECDHE-based):

ClientHello → ServerHello,

Certificate, (CertificateStatus/OCSP),

ServerKeyExchange (ECDHE params),

(CertificateRequest), ServerHelloDone

→ ClientKeyExchange (client's

ephemeral key), (Certificate,

CertificateVerify), ChangeCipherSpec,

Finished → Server ChangeCipherSpec,

Finished.

Security properties:

- Freshness: ClientRandom and ServerRandom ensure nonces per session; Finished MAC over transcript prevents replay.
- Server authentication: Certificate chain validates server's

identity; Finished MAC proves possession of the private key.

- Forward secrecy: Ephemeral DH keys ensure past sessions remain secure even if the long-term key is later compromised.

- Integrity and confidentiality: After ChangeCipherSpec, traffic is protected by negotiated AEAD on ENC+MAC keys.

Question 35

What is the general form of an elliptic curve equation over a finite field, and why is it used in cryptography?

Ans:

Over a prime field F_p ($p > 3$),

the short Weierstrass form is:

$$y^2 \equiv x^3 + ax + b \pmod{p}, \text{ with}$$

$$\text{discriminant } \Delta = -16(4a^3 + 27b^2) \neq 0$$

to avoid singularities.

The set of points (x, y) satisfying

the equation plus the point at

infinity forms an Abelian group

under a geometric chord-and-tangent

addition law.

Why used in cryptography: The

Elliptic Curve Discrete Logarithm

Problem (ECDLP) —given P and $Q =$

kP , find k —is believed to be hard,

allowing small key sizes with high

security. ECC provides efficient key exchange (ECDH), signatures (ECDSA /EdDSA), and performance advantages, especially on constrained devices.

Question 36

How does ECC achieve the same level of security as RSA with a smaller key size? Briefly explain.

Ans:

ECC's security is based on the ECDLP, for which the best-known classical attacks (e.g., Pollard's rho) have exponential complexity in $\sqrt{n}$, where n is the group order. In

contrast, RSA's hardness (integer factorization) has sub-exponential attacks (Number Field Sieve).

Consequently, ECC can use much smaller keys to achieve comparable security (e.g., ~ 256 -bit ECC $\approx$ 3072-bit RSA). Smaller keys mean faster computations, lower bandwidth, and reduced storage, benefiting many applications including mobile and IoT.

Question 37

Given the elliptic curve $y^2 \equiv x^3 + 2x + 3 \pmod{97}$, determine whether

the point $P = (3, 6)$ lies on the curve.

Ans:

Compute both sides modulo 97:

$$\text{LHS: } y^2 \equiv 6^2 = 36 \pmod{97}.$$

$$\begin{aligned} \text{RHS: } x^3 + 2x + 3 &\equiv 3^3 + 2 \cdot 3 + 3 = 27 + 6 \\ &+ 3 = 36 \pmod{97}. \end{aligned}$$

Since $\text{LHS} = \text{RHS}$, P lies on the curve.

Answer: Yes, $(3, 6)$ is on the curve.

Question 38

Given public key $(p = 23, g = 5, h = 8)$ and message $m = 10$, compute the

ElGamal ciphertext using random k
 $= 6$.

Ans:

ElGamal encryption outputs (c_1, c_2) :

$$c_1 = g^k \bmod p = 5^6 \bmod 23. \quad 5$$

$$2 = 2^5 \equiv 2; \quad 5^4 \equiv 2^2 = 4; \quad 5^6 \equiv 4 \times 2 = 8 \pmod{23}$$

$$23) \Rightarrow c_1 = 8.$$

$$c_2 = m \cdot h^k \bmod p. \text{ Compute } h^k =$$

$$8^6 \bmod 23:$$

$$8^2 = 64 \equiv 18; \quad 8^4 \equiv 18^2 = 324 \equiv 2; \quad 8^6 \equiv 8^4 \times 8$$

$$2 \equiv 2 \times 18 = 36 \equiv 13 \pmod{23}.$$

$$\text{Therefore } c_2 = 10 \times 13 \equiv 130 \equiv 15$$

$$\pmod{23}.$$

Ciphertext: $(c_1, c_2) = (8, 15)$.

Question 39

Explain how lightweight cryptography is important for securing IoT devices. Give one example of a lightweight encryption algorithm used in IoT.

Ans:

IoT devices often have limited CPU, memory, power, and bandwidth.

Lightweight cryptography delivers security with small code size, low RAM usage, and minimal energy consumption while maintaining acceptable latency. It enables secure boot, firmware updates, device-

to-cloud communication, and data integrity on constrained microcontrollers.

Example algorithms:

- ASCON (NIST Lightweight Cryptography winner): an authenticated encryption with associated data (AEAD) and hashing family designed for constrained devices, offering strong security with small footprint.
- Others include PRESENT , GIFT , Speck/Simon (historical /controversial), and CLEFIA . ASCON is widely recommended due to

standardization and robust design for embedded contexts.

Question 40

List and briefly explain the common IoT-specific attacks (e.g., firmware hijacking, physical tampering, botnets like Mirai). What mitigation strategies can be applied?

Ans:

Common IoT attacks:

- Weak/default credentials:

Attackers brute-force or use known defaults to take over devices.

- Firmware hijacking/supply-chain attacks: Malicious or altered

firmware implanted during manufacturing or updates.

- Physical tampering: Direct access to ports, debug interfaces (JTAG),

side-channel attacks, on chip-off for data extraction.

- Botnet recruitment (e.g., Mirai):

Mass compromise of insecure devices to launch DDoS attacks.

- Insecure update mechanisms: Lack of code signing allows arbitrary firmware installation.

- Insecure communications:

Plaintext or weakly encrypted protocols expose data and credentials.

- Inadequate isolation: Flat networks allow lateral movement and pivoting.

Mitigations:

- Strong authentication: Unique credentials, MFA where applicable, lockout and rate limiting.
- Secure boot and signed updates: Verify firmware authenticity and integrity; use immutable roots of trust.
- Hardening and least privilege: Disable unused services/ports; reduce attack surface; sandboxing.
- Physical security: Tamper-evident seals, secure enclosures, disable debug interfaces in production.
- Network defenses: Segment IoT from critical networks, use firewalling, IDS/IPS, and secure protocols (TLS/DTLS).

- Monitoring and asset management: Maintain inventories, anomaly detection, and rapid patching/rotation policies.
- Lifecycle management: Plan for secure decommissioning and credential/key revocation.